T3 Optimización y cálculo matricial

Índice

1. Introducción
2. Derivada
3. Gradiente
4. Jacobiana
5. Descenso por gradiente
6. Cálculo matricial
7. Optimización con restricciones

1 Introducción

Objetivo principal: introducir conceptos básicos para aplicar descenso por gradiente estocástico

Derivada, gradiente y Jacobiana: conceptos básicos muy usados

Descenso por gradiente estocástico: algoritmo de optimización estándar en aprendizaje automático

Cálculo matricial: notación especializada para trabajar con Jacobianas cómodamente

Optimización con restricciones: el conjunto de soluciones posibles suele restringirse con igualdades y desigualdades

2 Derivada

Función de $\mathbb{R}$ en $\mathbb{R}$: $f : \mathbb{R} \to \mathbb{R}, \quad y = f(x)$

Derivada de f: $f'(x) = \lim_{h \to 0} \dfrac{f(x+h) - f(x)}{h}$

Interpretación geométrica: pendiente de la recta tangente en $f(x)$, $f(x+h) \approx f(x) + f'(x)\,h$

Aproximación de diferencia finita: dado un $h > 0$ pequeño, $f'(x) \approx \dfrac{f(x+h) - f(x)}{h}$

La diferenciación es un operador funcional: $D(f) = f'$

Notación de Lagrange: f' denota la derivada; f'' la segunda; $f^{(n)}$ la n-ésima

Notación de Leibinz: $\dfrac{dy}{dx}$ o $\dfrac{d}{dx} f(x)$ denota la derivada y $\left.\dfrac{df}{dx}\right|_{x=a}$ la derivada en un punto a

Neg-derivada: derivada con signo cambiado

- Si es positiva, $f(x)$ decrece en la dirección de x (al aumentar x)
- Si es negativa, $f(x)$ crece en la dirección de x (al aumentar x)

Punto estacionario: punto de derivada nula

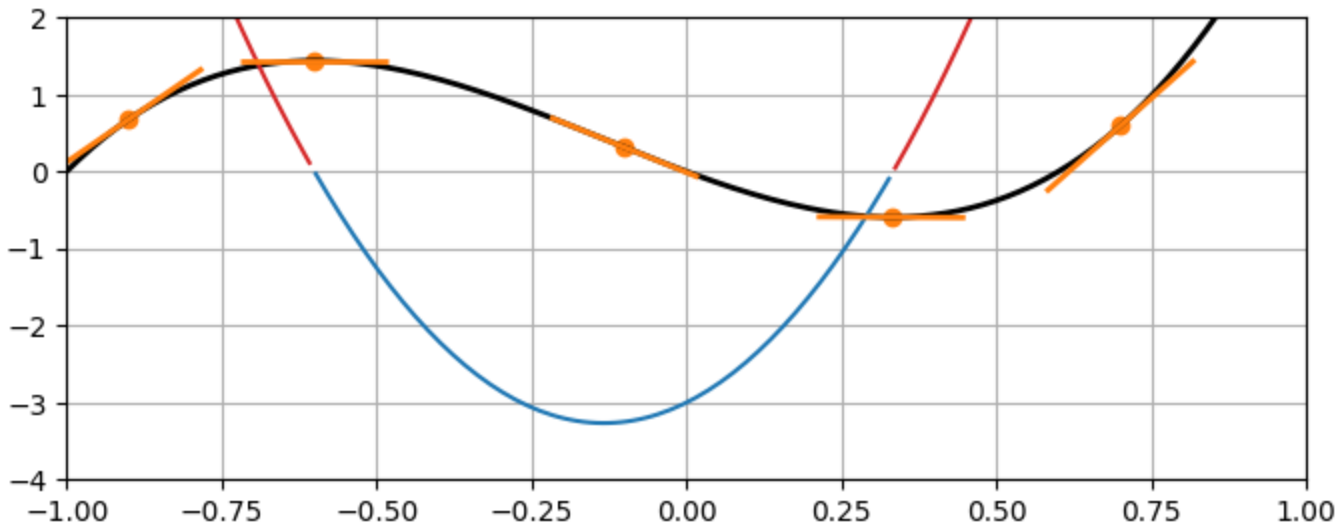
Tipos de puntos estacionarios: mínimo si la segunda derivada es positiva; máximo si es negativa

Ejercicio: Halla la derivada de $f(x) = 5x^3 + 2x^2 - 3x$, puntos estacionarios y tipo de cada uno

Solución: $f(x) = 5x^3 + 2x^2 - 3x$

$$f'(x) = 15x^2 + 4x - 3, \quad f'(x) = 0 \to x_1 = -3/5,\ x_2 = 1/3$$
$$f''(x) = 30x + 4, \quad f''(x_1) = -14 < 0 \to \text{máximo}, \quad f''(x_2) = 14 > 0 \to \text{mínimo}$$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from matplotlib.collections import LineCollection
f = lambda x: 5 * np.power(x, 3) + 2 * np.square(x) - 3 * x; d = lambda x: 15 * np.square(x) + 4 * x - 3
fig, ax = plt.subplots(1, 1, figsize=(8, 3)); ax.grid(); ax.axis([-1, 1, -4, 2])
x = np.linspace(-1, 1, 200); plt.plot(x, f(x), c='black', lw=2)
plt.plot(x, np.ma.masked_less(d(x), 0), c='C3')
plt.plot(x, np.ma.masked_greater_equal(d(x), 0), c='C0')
X = np.array([-.9, -.6, -.1, .33, .7]); Y = f(X); ax.scatter(X, Y, c='C1')
L = np.zeros((len(X), 2, 2)); r = 0.12
L[:, 0, :] = np.c_[X - r, Y - r * d(X)]
L[:, 1, :] = np.c_[X + r, Y + r * d(X)]
ax.add_collection(LineCollection(L, lw=2, colors='C1', ls='solid'));
```



3 Gradiente

Función de $\mathbb{R}^n$ en $\mathbb{R}$: $f : \mathbb{R}^{\mathrm{n}} \to \mathbb{R}, \quad y = f(\boldsymbol{x})$

Derivada parcial: $$\frac{\partial f}{\partial x_i} = \lim_{h \to 0} \frac{f(\boldsymbol{x} + h\boldsymbol{e}_i) - f(\boldsymbol{x})}{h} \qquad (\boldsymbol{e}_i \text{ es el i-ésimo vector unitario})$$

Gradiente de f: vector de derivadas parciales

$$\boldsymbol{g} = \frac{\partial f}{\partial \boldsymbol{x}} = \nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \qquad \text{o} \qquad \boldsymbol{g}(\boldsymbol{x}^*) = \left.\frac{\partial f}{\partial \boldsymbol{x}}\right|_{\boldsymbol{x}^*} \quad (\text{enfatiza que se evalúa en } \boldsymbol{x}^*)$$

Neg-gradiente: gradiente con signo cambiado; indica la dirección en la que f decrece más rápidamente

Punto estacionario: punto de gradiente nulo

Hessiana: matriz de derivadas parciales segundas

$$\mathbf{H}(\boldsymbol{x}) = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \begin{pmatrix} \frac{\partial g_1}{\partial \boldsymbol{x}} \\ \vdots \\ \frac{\partial g_n}{\partial \boldsymbol{x}} \end{pmatrix} = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \vdots & \vdots & \vdots \\ \frac{\partial g_n}{\partial x_1} & \cdots & \frac{\partial g_n}{\partial x_n} \end{pmatrix}$$

Tipos de puntos estacionarios: mínimo si la Hessiana es definida positiva; máximo si es definida negativa

Ejercicio: Halla el gradiente de $y = f(\boldsymbol{x}) = a_1 x_1^2 + a_2 x_2^2, \quad a_1, a_2 \neq 0,$ puntos estacionarios y tipo de cada uno

Solución: $\boldsymbol{x} = (x_1, x_2)^t, \quad y = f(\boldsymbol{x}) = a_1 x_1^2 + a_2 x_2^2, \quad a_1, a_2 \neq 0$

$$\boldsymbol{g}(\boldsymbol{x}) = \frac{\partial y}{\partial \boldsymbol{x}} = \begin{pmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \end{pmatrix} = \begin{pmatrix} 2a_1 x_1 \\ 2a_2 x_2 \end{pmatrix}$$

$$\boldsymbol{g}(\boldsymbol{x}) = \mathbf{0} \;\rightarrow\; \boldsymbol{x} = \mathbf{0}$$

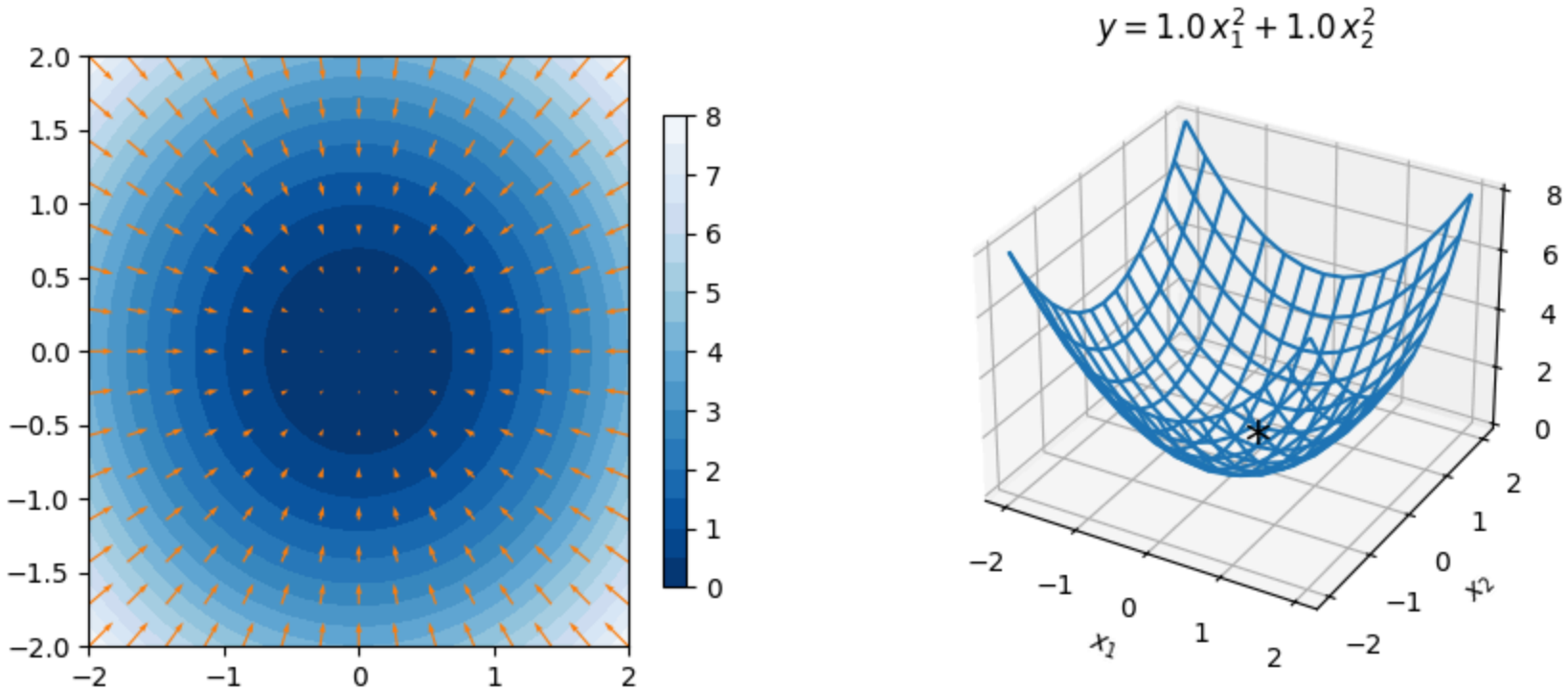
$$\mathbf{H}(\boldsymbol{x}) = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \begin{pmatrix} \frac{\partial g_1}{\partial \boldsymbol{x}} \\ \frac{\partial g_2}{\partial \boldsymbol{x}} \end{pmatrix} = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} \end{pmatrix}$$

$$= \operatorname{diag}(2a_1, 2a_2) = \begin{cases} \succ 0 & \text{si } a_1, a_2 > 0 \\ \prec 0 & \text{si } a_1, a_2 < 0 \\ \text{indefinida} & a_1, a_2 \text{ de signo opuesto} \end{cases}$$

$$\boldsymbol{x} = \mathbf{0} \rightarrow \begin{cases} \text{mínimo} & \text{si } a_1, a_2 > 0 \\ \text{máximo} & \text{si } a_1, a_2 < 0 \\ \text{punto de silla} & \text{si } a_1, a_2 \text{ de signo opuesto} \end{cases}$$

Solución (cont.): ejemplo con a_1 y a_2 positivos; prueba ambos negativos y también de signo opuesto

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
R = np.linspace(-2, 2, 50); X1, X2 = np.meshgrid(R, R); X = np.c_[np.ravel(X1), np.ravel(X2)]
a = np.array([1, 1]); Y = (np.square(X) @ a).reshape(X1.shape)
fig = plt.figure(figsize=(10, 4)); ax = plt.subplot(121)
cp = ax.contourf(X1, X2, Y, 15, cmap='Blues_r'); plt.colorbar(cp, ax=ax, shrink=.8)
r = np.linspace(-2, 2, 15); x1, x2 = np.meshgrid(r, r); x = np.c_[np.ravel(x1), np.ravel(x2)]
neg_grad = -x @ np.diag(2 * a); ax.quiver(x[:, 0], x[:, 1], neg_grad[:, 0], neg_grad[:, 1], color='C1');
ax = plt.subplot(122, projection='3d'); ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
ax.set_title(f'$y={a[0]:.1f}\,x_1^2{a[1]:+.1f}\,x_2^2$'); ax.text(0, 0, 0, '*', fontsize=20)
ax.plot_wireframe(x1, x2, a[0]*x1**2 + a[1]*x2**2);
```



4 Jacobiana

Función de $\mathbb{R}^n$ en $\mathbb{R}^m$: $\boldsymbol{f} : \mathbb{R}^{\mathrm{n}} \to \mathbb{R}^m, \quad \boldsymbol{y} = \boldsymbol{f}(\boldsymbol{x}) = (f_1(\boldsymbol{x}), \ldots, f_m(\boldsymbol{x}))^t$

Jacobiana de $\boldsymbol{f}$: matriz de derivadas parciales

$$\mathbf{J} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial \boldsymbol{x}} \\ \vdots \\ \frac{\partial f_m}{\partial \boldsymbol{x}} \end{pmatrix} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \vdots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\boldsymbol{x})^t \\ \vdots \\ \nabla f_m(\boldsymbol{x})^t \end{pmatrix}$$

Jacobiana de una función de $\mathbb{R}^n$ en $\mathbb{R}$: es su gradiente transpuesto

Jacobiana del gradiente de una función de $\mathbb{R}^n$ en $\mathbb{R}$: es su Hessiana

$$\mathbf{J} = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \begin{pmatrix} \frac{\partial g_1}{\partial \boldsymbol{x}} \\ \vdots \\ \frac{\partial g_n}{\partial \boldsymbol{x}} \end{pmatrix} = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \vdots & \vdots & \vdots \\ \frac{\partial g_n}{\partial x_1} & \cdots & \frac{\partial g_n}{\partial x_n} \end{pmatrix} = \mathbf{H}(\boldsymbol{x})$$

Ejercicio: Halla la Jacobiana de $\boldsymbol{y} = \mathbf{W}\boldsymbol{x},\ \boldsymbol{x} \in \mathbb{R}^n,\ \mathbf{W} \in \mathbb{R}^{m \times n},\ m = 2,\ n = 3$

Solución: $\boldsymbol{y} = \mathbf{W}\boldsymbol{x},\ \boldsymbol{x} \in \mathbb{R}^n,\ \mathbf{W} \in \mathbb{R}^{m\times n},\ m = 2,\ n = 3$

$$\begin{aligned}\begin{pmatrix} y_1 \\ y_2 \end{pmatrix} &= \begin{pmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \\ &= \begin{pmatrix} W_{11}x_1 + W_{12}x_2 + W_{13}x_3 \\ W_{21}x_1 + W_{22}x_2 + W_{23}x_3 \end{pmatrix}\end{aligned}$$

$$\mathbf{J} = \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \begin{bmatrix} \dfrac{\partial y_1}{\partial \boldsymbol{x}} \\ \dfrac{\partial y_2}{\partial \boldsymbol{x}} \end{bmatrix} = \begin{bmatrix} \dfrac{\partial y_1}{\partial x_1} & \dfrac{\partial y_1}{\partial x_2} & \dfrac{\partial y_1}{\partial x_3} \\ \dfrac{\partial y_2}{\partial x_1} & \dfrac{\partial y_2}{\partial x_2} & \dfrac{\partial y_2}{\partial x_3} \end{bmatrix} = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix} = \mathbf{W}$$

5 Descenso por gradiente

Objetivo: minimizar $f : \mathbb{R}^n \to \mathbb{R}$ iterativamente, a partir de un punto de inicio $\boldsymbol{x}_0$

Descenso por gradiente (GD): escoge la dirección de descenso más pronunciada, dada por el neg-gradiente

$$\boldsymbol{x}_{i+1} = \boldsymbol{x}_i - \eta_i \boldsymbol{g}(\boldsymbol{x}_i) \quad \text{donde} \quad \eta_i > 0 \qquad (i = 0, 1, \ldots)$$

Factor de aprendizaje constante: $\eta_i = \eta$; GD puede no converger o hacerlo muy lentamente

Caída escalonada: $\eta_i = \eta_0 \gamma^{\lfloor i/s \rfloor}$ con *decay rate* γ y *decay step* s; por ejemplo, $\eta_0 = 0.1$, $\gamma = 0.5$ y $s = 10$

Momentum: acelera el movimiento en regiones llanas del objetivo (y lo frena en abruptas)

$$\begin{aligned} \boldsymbol{x}_{i+1} &= \boldsymbol{x}_i - \eta\, \boldsymbol{m}_{i+1} \\ \boldsymbol{m}_{i+1} &= \beta \boldsymbol{m}_i + \boldsymbol{g}(\boldsymbol{x}_i) \quad \text{donde} \quad \beta \in [0, 1) \quad (\text{típicamente}\, \beta = 0.9) \end{aligned}$$

Inconveniente: momentum suele oscilar al acercarse a un mínimo

Momentum Nesterov: añade un paso de extrapolación para amortiguar oscilaciones

$$\begin{aligned} \boldsymbol{x}_{i+1} &= \boldsymbol{x}_i + \boldsymbol{m}_{i+1} \\ \boldsymbol{m}_{i+1} &= \beta \boldsymbol{m}_i - \eta\, \boldsymbol{g}(\boldsymbol{x}_i + \beta \boldsymbol{m}_i) \quad \text{donde} \quad \beta \in [0, 1) \quad (\text{típicamente}\, \beta = 0.9) \end{aligned}$$

Implementación: GD con caída escalonada y momentum Nesterov; devuelve el camino recorrido

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from matplotlib.collections import LineCollection
```

In [ ]:
```
def GD(g, x0, eta=0.1, decay_rate=0.5, decay_step=10, beta=0.9, tol=1e-3, max_iter=100000):
    n = len(x0); X = np.zeros((max_iter + 1, n)); X[0, :] = x0
    delta = np.inf; i = 0; m = np.zeros(2)
    while np.max(np.abs(delta)) > tol and i < max_iter:
        m = beta * m - eta * g(X[i, :] + beta * m)
        delta = m
        X[i+1, :] = X[i, :] + delta
        i = i + 1
        if i % decay_step == 0: eta = eta * decay_rate
    return X[:i+1, :]
```

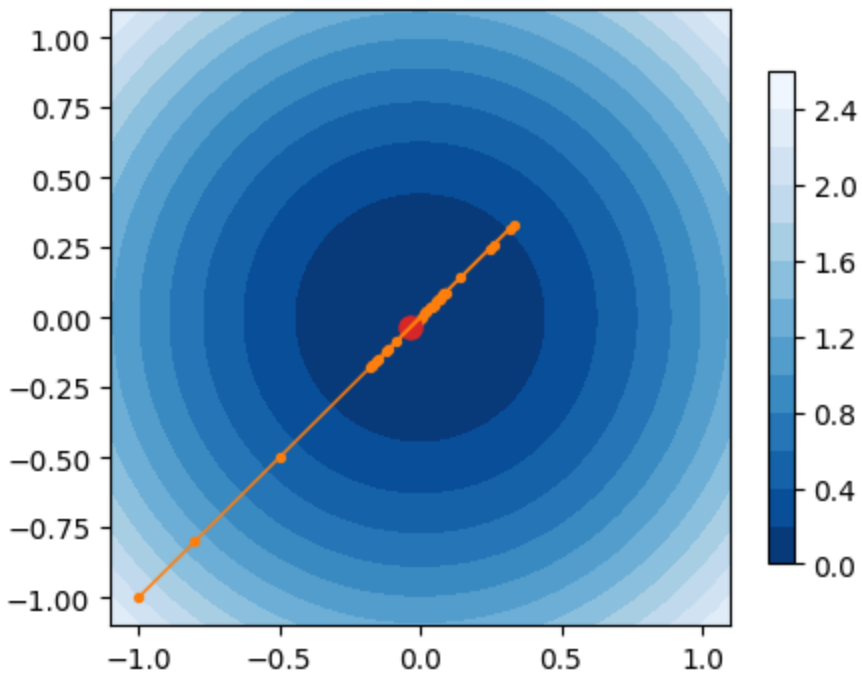
Función auxiliar: para mostrar el camino recorrido por GD en 2d

In [ ]:
```
def plot_GD(f, P, figsize=(5, 4), xlim=(-1.1, 1.1), ylim=(-1.1, 1.1), levels=15, cmap='Blues_r'):
    X1, X2 = np.meshgrid(np.linspace(*xlim, 50), np.linspace(*ylim, 50))
    X = np.c_[np.ravel(X1), np.ravel(X2)]
    Y = np.apply_along_axis(f, 1, X).reshape(X1.shape)
    _, ax = plt.subplots(1, 1, figsize=figsize)
    ax.set_xlim(xlim); ax.set_ylim(ylim)
    cp = ax.contourf(X1, X2, Y, levels=levels, cmap=cmap)
    plt.colorbar(cp, ax=ax, shrink=.8)
    ax.scatter(*P.T, c='C1', s=8)
    ax.scatter(*P[-1, :].T, c='C3', s=64)
    L = np.zeros((len(P)-1, 2, 2))
    L[:, 0, :] = P[:-1, :]; L[:, 1, :] = P[1:, :]
    ax.add_collection(LineCollection(L, lw=1, colors='C1', ls='solid'));
```

Ejemplo (cont.): Minimización de $y = a_1 x_1^2 + a_2 x_2^2$, $a_1 = a_2 = 1$, a partir de $\boldsymbol{x}_0 = (-1, -1)^t$

In [ ]:
```
a = np.array([1, 1]); f = lambda x: (a @ np.square(x)).sum(); g = lambda x: x @ np.diag(2 * a)
P = GD(g, np.array([-1, -1])); plot_GD(f, P)
print(f'iteraciones = {len(P)-1}  solución = {np.round(P[-1, :], 4)}')
```

```
iteraciones = 45  solución = [-0.0337 -0.0337]
```



Ejemplo (cont.): convergencia y número de iteraciones de GD en función del learning rate y la β de Nesterov

1. GD estándar ($\beta = 0$): ¿Converge con factor de aprendizaje constante muy pequeño o muy grande?
2. GD estándar ($\beta = 0$): ¿Converge con caída escalonada muy rápida o muy lenta?
3. Momentum Nesterov: Si la caída escalonada se ajusta adecuadamente, ¿cuál es el efecto de β?

In [ ]:
```
for eta in [1e-3, 1e-2, 1e-1, 1e0]:
    P = GD(g, np.array([-1, -1]), eta=eta, decay_rate=1.0, beta=0)
    print(f'eta = {eta}  iteraciones = {len(P)-1}  solución = {np.round(P[-1, :], 4)}')
```

```
eta = 0.001  iteraciones = 348  solución = [-0.4982 -0.4982]
eta = 0.01  iteraciones = 150  solución = [-0.0483 -0.0483]
eta = 0.1  iteraciones = 25  solución = [-0.0038 -0.0038]
eta = 1.0  iteraciones = 100000  solución = [-1. -1.]
```

In [ ]:
```
for rate in [.01, .1, .5]:
    for step in [1, 10, 50]:
        P = GD(g, np.array([-1, -1]), decay_rate=rate, decay_step=step,  beta=0)
        print(f'rate = {rate:.2f}  step = {step:2d}  iteraciones = {len(P)-1:3d}  solución = {P[-1, :]}')
```

```
rate = 0.01  step =  1  iteraciones =   3  solución = [-0.79838403 -0.79838403]
rate = 0.01  step = 10  iteraciones =  11  solución = [-0.10715943 -0.10715943]
rate = 0.01  step = 50  iteraciones =  25  solución = [-0.00377789 -0.00377789]
rate = 0.10  step =  1  iteraciones =   4  solución = [-0.78227551 -0.78227551]
rate = 0.10  step = 10  iteraciones =  21  solución = [-0.08755706 -0.08755706]
rate = 0.10  step = 50  iteraciones =  25  solución = [-0.00377789 -0.00377789]
rate = 0.50  step =  1  iteraciones =   9  solución = [-0.65087431 -0.65087431]
rate = 0.50  step = 10  iteraciones =  31  solución = [-0.02185575 -0.02185575]
rate = 0.50  step = 50  iteraciones =  25  solución = [-0.00377789 -0.00377789]
```

In [ ]:
```
for beta in [.1, .5, .9]:
    P = GD(g, np.array([-1, -1]), beta=beta)
    print(f'beta = {beta}  iteraciones = {len(P)-1}  solución = {np.round(P[-1, :], 4)}')
```

```
beta = 0.1  iteraciones = 27  solución = [-0.0167 -0.0167]
beta = 0.5  iteraciones = 10  solución = [0.011 0.011]
beta = 0.9  iteraciones = 45  solución = [-0.0337 -0.0337]
```

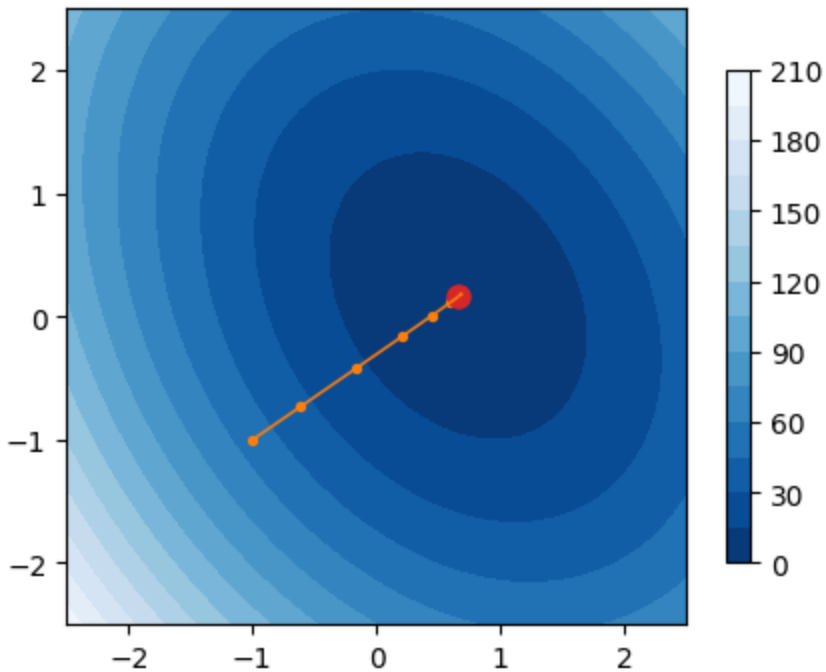
Ejemplo: número de iteraciones de GD con una función cuadrática de matriz bien condicionada

$$y = \tfrac{1}{2}\boldsymbol{x}^t\mathbf{A}\boldsymbol{x} + \boldsymbol{b}^t\boldsymbol{x} + c, \quad \mathbf{A} = \begin{pmatrix} 20 & 5 \\ 5 & 16 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} -14 \\ -6 \end{pmatrix}, \quad c = 10, \quad \boldsymbol{x}^* = -2\,(\mathbf{A} + \mathbf{A}^t)^{-1}\boldsymbol{b}$$

$$\boldsymbol{x}_0 = (-1, -1)^t, \quad \boldsymbol{g}(\boldsymbol{x}) = \tfrac{1}{2}(\mathbf{A} + \mathbf{A}^t)\boldsymbol{x} + \boldsymbol{b}$$

In [ ]:
```
A, b, c = np.array([ [20, 5], [5, 16] ]), np.array([-14, -6]), 10; xstar = -2 * np.linalg.inv(A+A.T) @ b
f = lambda x: 0.5 * x.T @ A @ x + b @ x + c; g = lambda x: 0.5 * (A + A.T) @ x + b
P = GD(g, np.array([-1, -1]), eta=0.01, beta=0.5, tol=np.finfo(float).eps)
plot_GD(f, P, xlim=(-2.5, 2.5), ylim=(-2.5, 2.5))
print(f'min = {np.round(xstar, 4)}  i = {len(P)-1}  sol = {np.round(P[-1, :], 4)}')
```

```
min = [0.6576 0.1695]  i = 361  sol = [0.6577 0.1695]
```



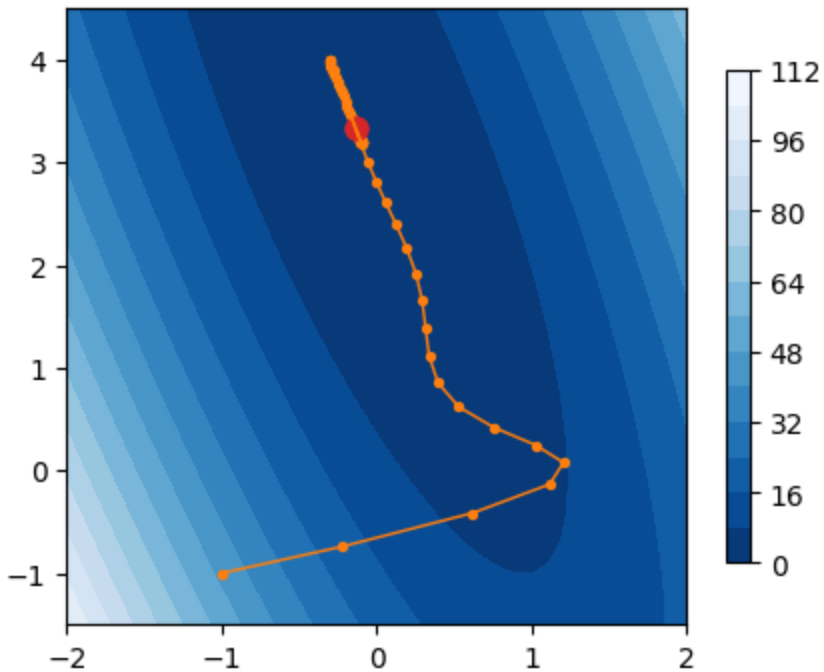
Ejemplo (cont.): número de iteraciones de GD con una función cuadrática de matriz peor condicionada

$$y = \tfrac{1}{2}\boldsymbol{x}^t\mathbf{A}\boldsymbol{x} + \boldsymbol{b}^t\boldsymbol{x} + c, \quad \mathbf{A} = \begin{pmatrix} 20 & 5 \\ 5 & 2 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} -14 \\ -6 \end{pmatrix}, \quad c = 10, \quad \boldsymbol{x}^* = -2\,(\mathbf{A} + \mathbf{A}^t)^{-1}\boldsymbol{b}$$

$$\boldsymbol{x}_0 = (-1, -1)^t, \quad \boldsymbol{g}(\boldsymbol{x}) = \tfrac{1}{2}(\mathbf{A} + \mathbf{A}^t)\boldsymbol{x} + \boldsymbol{b}$$

In [ ]:
```
A, b, c = np.array([ [20, 5], [5,  2] ]), np.array([-14, -6]), 10; xstar = -2 * np.linalg.inv(A+A.T) @ b
f = lambda x: 0.5 * x.T @ A @ x + b @ x + c; g = lambda x: 0.5 * (A + A.T) @ x + b
P = GD(g, np.array([-1, -1]), eta=0.02, decay_step=50, beta=0.9, tol=np.finfo(float).eps)
plot_GD(f, P, xlim=(-2, 2), ylim=(-1.5, 4.5))
print(f'min = {np.round(xstar, 4)}  i = {len(P)-1}  sol = {np.round(P[-1, :], 4)}')
```

```
min = [-0.1333  3.3333]  i = 1867  sol = [-0.1334  3.3335]
```



Diferenciación simbólica: diferenciación mediante computación simbólica

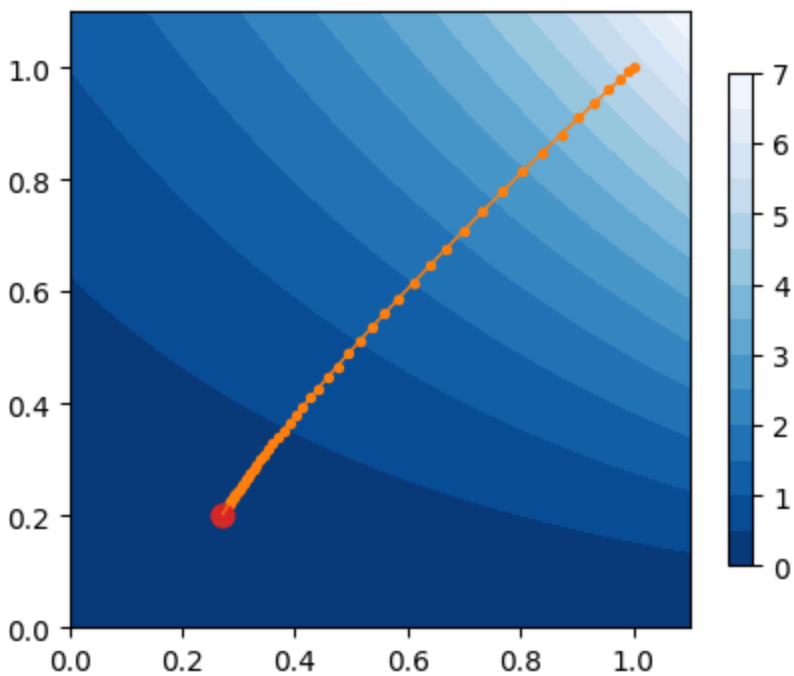
SymPy: librería python de computación simbólica; ver https://www.sympy.org

Ejemplo: $f(x_1, x_2) = x_2 \exp(x_1)\sqrt{x_1 + x_2 \exp(x_1)}$

In [ ]:
```
from sympy import symbols, exp, sqrt, simplify, lambdify
x, y = symbols('x1 x2')
spf = y * sqrt(x + y * exp(x)) * exp(x); f = lambdify([x, y], spf, "numpy")
spdx = simplify(spf.diff(x)); dx = lambdify([x, y], spdx, "numpy")
spdy = simplify(spf.diff(y)); dy = lambdify([x, y], spdy, "numpy") # display(spf, spdx, spdy)
```

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
npf = lambda xy: f(xy[0], xy[1])
npg = lambda xy: np.array([dx(xy[0], xy[1]), dy(xy[0], xy[1])])
P = GD(npg, np.array([1, 1]), eta=.001); plot_GD(npf, P, xlim=(0, 1.1), ylim=(0, 1.1))
print(f'iteraciones = {len(P)-1}  solución = {np.round(P[-1, :], 4)}')
```

```
iteraciones = 61  solución = [0.2712 0.2006]
```



Función (valle o banana de) Rosenbrock: mínimo global en (a, a^2); https://en.wikipedia.org/wiki/Rosenbrock_function

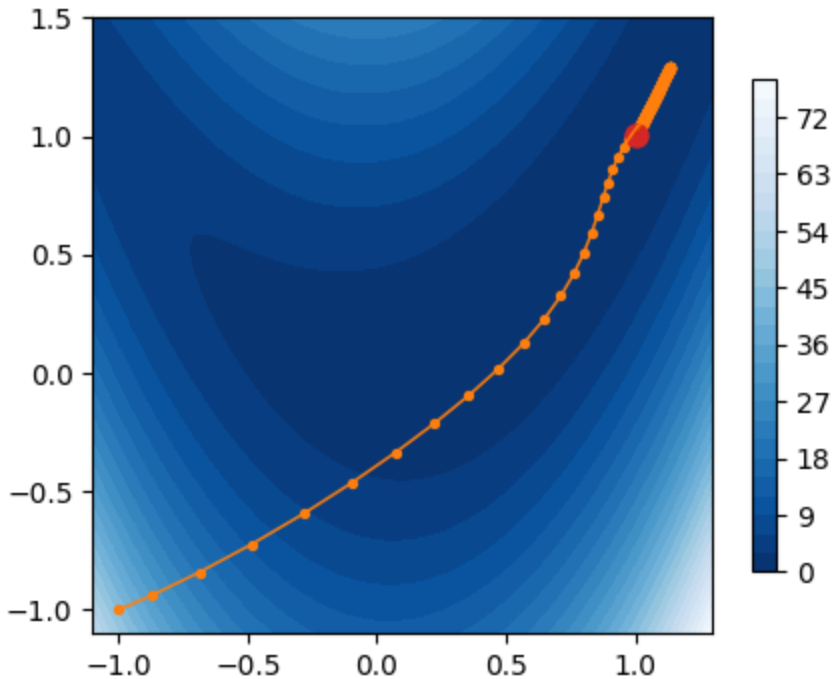
$$f(x,y) = (a-x)^2 + b(y-x^2)^2$$

Ejemplo: GD aplicado a Rosenbrock con $a = 1,\ b = 10$ y punto de inicio $(-1,-1)^t$

In [ ]:
```
from sympy import symbols, simplify, lambdify
x, y = symbols('x1 x2')
spf = (1-x)**2 + 10 * (y-x**2)**2; f = lambdify([x, y], spf, "numpy")
spdx = simplify(spf.diff(x)); dx = lambdify([x, y], spdx, "numpy")
spdy = simplify(spf.diff(y)); dy = lambdify([x, y], spdy, "numpy") # display(spf, spdx, spdy)
```

In [ ]:
```
npf = lambda xy: f(xy[0], xy[1]); npg = lambda xy: np.array([dx(xy[0], xy[1]), dy(xy[0], xy[1])])
tol = np.finfo(float).eps
P = GD(npg, np.array([-1, -1]), eta=0.0015, decay_step=100, decay_rate=0.9, beta=0.9, tol=np.finfo(float).eps)
plot_GD(npf, P, xlim=(-1.1, 1.3), ylim=(-1.1, 1.5), levels=30, cmap='Blues_r')
print(f'i = {len(P)-1}  sol = {np.round(P[-1, :], 4)}')
```

```
i = 22918  sol = [1.0005 1.0011]
```



6 Cálculo matricial

Cálculo matricial: notación especializada para manejar derivadas parciales (ver Matrix_calculus en wikipedia)

Formato numerador: primero el del numerador y luego el del denominador transpuesto, **si cabe en una matriz**

Tipos	Escalar $y \in \mathbb{R}^{1\times 1}$	Vector $\boldsymbol{y} \in \mathbb{R}^{m\times 1}$	Matriz $\mathbf{Y} \in \mathbb{R}^{m\times n}$
$x \in \mathbb{R}^{1\times 1}$	$\dfrac{\partial y}{\partial x}$	$\dfrac{\partial \boldsymbol{y}}{\partial x} = \begin{bmatrix} \dfrac{\partial y_1}{\partial x} \\ \vdots \\ \dfrac{\partial y_m}{\partial x} \end{bmatrix}$	$\dfrac{\partial \mathbf{Y}}{\partial x} = \begin{bmatrix} \dfrac{\partial y_{11}}{\partial x} & \cdots & \dfrac{\partial y_{1n}}{\partial x} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial y_{m1}}{\partial x} & \cdots & \dfrac{\partial y_{mn}}{\partial x} \end{bmatrix}$
$\boldsymbol{x} \in \mathbb{R}^{n\times 1}$	$\dfrac{\partial y}{\partial \boldsymbol{x}} = \begin{bmatrix} \dfrac{\partial y}{\partial x_1} & \cdots & \dfrac{\partial y}{\partial x_n} \end{bmatrix}$	$\dfrac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \begin{bmatrix} \dfrac{\partial y_1}{\partial \boldsymbol{x}} \\ \vdots \\ \dfrac{\partial y_m}{\partial \boldsymbol{x}} \end{bmatrix} = \begin{bmatrix} \dfrac{\partial y_1}{\partial x_1} & \cdots & \dfrac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial y_m}{\partial x_1} & \cdots & \dfrac{\partial y_m}{\partial x_n} \end{bmatrix}$	$\dfrac{\partial \mathbf{Y}}{\partial \boldsymbol{x}} = \begin{bmatrix} \dfrac{\partial y_{11}}{\partial \boldsymbol{x}} & \cdots & \dfrac{\partial y_{1n}}{\partial \boldsymbol{x}} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial y_{m1}}{\partial \boldsymbol{x}} & \cdots & \dfrac{\partial y_{mn}}{\partial \boldsymbol{x}} \end{bmatrix}$ 3T
$\mathbf{X} \in \mathbb{R}^{p\times q}$	$\dfrac{\partial y}{\partial \mathbf{X}} = \begin{bmatrix} \dfrac{\partial y}{\partial x_{11}} & \cdots & \dfrac{\partial y}{\partial x_{p1}} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial y}{\partial x_{1q}} & \cdots & \dfrac{\partial y}{\partial x_{pq}} \end{bmatrix}$	$\dfrac{\partial \boldsymbol{y}}{\partial \mathbf{X}} = \begin{bmatrix} \dfrac{\partial y_1}{\partial \mathbf{X}} \\ \vdots \\ \dfrac{\partial y_m}{\partial \mathbf{X}} \end{bmatrix}$ 3T	$\dfrac{\partial \mathbf{Y}}{\partial \mathbf{X}} = \begin{bmatrix} \dfrac{\partial y_{11}}{\partial \mathbf{X}} & \cdots & \dfrac{\partial y_{1n}}{\partial \mathbf{X}} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial y_{m1}}{\partial \mathbf{X}} & \cdots & \dfrac{\partial y_{mn}}{\partial \mathbf{X}} \end{bmatrix}$ 4T

Casos de interés: los de la columna izquierda y centro . . . qué hacemos con el 3-tensor $\dfrac{\partial \boldsymbol{y}}{\partial \mathbf{X}} \in \mathbb{R}^{m\times q\times p}$?

Caso de interés especial:

- En general no necesitaremos hacer cálculos con el 3-tensor $\dfrac{\partial \boldsymbol{y}}{\partial \mathbf{X}} \in \mathbb{R}^{m\times q\times p}$
- Sí necesitaremos calcular $\boldsymbol{u}^t \dfrac{\partial \boldsymbol{y}}{\partial \mathbf{X}}$ con $\boldsymbol{u}^t \in \mathbb{R}^{1\times m}$ constante
- $\boldsymbol{u}^t \dfrac{\partial \boldsymbol{y}}{\partial \mathbf{X}} \in \mathbb{R}^{1\times q\times p}$ con un único elemento $\mathbb{R}^{q\times p}$ en la primera dimensión, que extraemos por simplicidad

$$\boldsymbol{u}^t \frac{\partial \boldsymbol{y}}{\partial \mathbf{X}} = [u_1, \ldots, u_m] \begin{bmatrix} \dfrac{\partial y_1}{\partial \mathbf{X}} \\ \vdots \\ \dfrac{\partial y_m}{\partial \mathbf{X}} \end{bmatrix} = \left[\sum_{i=1}^{m} u_i \frac{\partial y_i}{\partial \mathbf{X}}\right] \quad \rightarrow \quad \left[\boldsymbol{u}^t \frac{\partial \boldsymbol{y}}{\partial \mathbf{X}}\right]_{1,:,:} = \sum_{i=1}^{m} u_i \frac{\partial y_i}{\partial \mathbf{X}}$$

Formato denominador: primero el del denominador y luego el del numerador transpuesto, **si cabe en una matriz**

- El resultado es el mismo que en formato numerador, aunque transpuesto

Origen de los dos formatos: se halla, sobre todo, en el formato que se da a $\dfrac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}}$

Formato mixto: consiste en escribir $\dfrac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}^t}$ y seguir el formato numerador

Recomendación para evitar errores: usar siempre el mismo formato, numerador si puede ser

Identidades para escalares: a constante; u y v funciones reales de $x \in \mathbb{R}^{1\times 1}$, $\boldsymbol{x} \in \mathbb{R}^{n\times 1}$ o $\mathbf{X} \in \mathbb{R}^{p\times q}$

$y \in \mathbb{R}^{1\times 1}$	$\dfrac{\partial y}{\partial x} \in \mathbb{R}^{1\times 1}$	$\dfrac{\partial y}{\partial \boldsymbol{x}} \in \mathbb{R}^{1\times n}$	$\dfrac{\partial y}{\partial \mathbf{X}} \in \mathbb{R}^{q\times p}$
a	0	$\mathbf{0}$	$\mathbf{0}$
au	$a\dfrac{\partial u}{\partial x}$	$a\dfrac{\partial u}{\partial \boldsymbol{x}}$	$a\dfrac{\partial u}{\partial \mathbf{X}}$
$u+v$	$\dfrac{\partial u}{\partial x}+\dfrac{\partial v}{\partial x}$	$\dfrac{\partial u}{\partial \boldsymbol{x}}+\dfrac{\partial v}{\partial \boldsymbol{x}}$	$\dfrac{\partial u}{\partial \mathbf{X}}+\dfrac{\partial v}{\partial \mathbf{X}}$
uv	$u\dfrac{\partial v}{\partial x}+v\dfrac{\partial u}{\partial x}$	$u\dfrac{\partial v}{\partial \boldsymbol{x}}+v\dfrac{\partial u}{\partial \boldsymbol{x}}$	$u\dfrac{\partial v}{\partial \mathbf{X}}+v\dfrac{\partial u}{\partial \mathbf{X}}$
$g(u)$	$\dfrac{\partial g(u)}{\partial u}\dfrac{\partial u}{\partial x}$	$\dfrac{\partial g(u)}{\partial u}\dfrac{\partial u}{\partial \boldsymbol{x}}$	$\dfrac{\partial g(u)}{\partial u}\dfrac{\partial u}{\partial \mathbf{X}}$
$f(g(u))$	$\dfrac{\partial f(g)}{\partial g}\dfrac{\partial g(u)}{\partial u}\dfrac{\partial u}{\partial x}$	$\dfrac{\partial f(g)}{\partial g}\dfrac{\partial g(u)}{\partial u}\dfrac{\partial u}{\partial \boldsymbol{x}}$	$\dfrac{\partial f(g)}{\partial g}\dfrac{\partial g(u)}{\partial u}\dfrac{\partial u}{\partial \mathbf{X}}$

Identidades para vectores: $\boldsymbol{a}$ constante; $\boldsymbol{u}$ y $\boldsymbol{v}$ funciones vectoriales de $x \in \mathbb{R}^{1\times 1}$ o $\boldsymbol{x} \in \mathbb{R}^{n\times 1}$

$\boldsymbol{y} \in \mathbb{R}^{m\times 1}$	$\dfrac{\partial \boldsymbol{y}}{\partial x} \in \mathbb{R}^{m\times 1}$	$\dfrac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} \in \mathbb{R}^{m\times n}$
$\boldsymbol{a}$	$\mathbf{0}$	$\mathbf{0}$
$a\boldsymbol{u}$	$a\dfrac{\partial \boldsymbol{u}}{\partial x}$	$a\dfrac{\partial \boldsymbol{u}}{\partial \boldsymbol{x}}$
$\boldsymbol{u}+\boldsymbol{v}$	$\dfrac{\partial \boldsymbol{u}}{\partial x}+\dfrac{\partial \boldsymbol{v}}{\partial x}$	$\dfrac{\partial \boldsymbol{u}}{\partial \boldsymbol{x}}+\dfrac{\partial \boldsymbol{v}}{\partial \boldsymbol{x}}$
$\boldsymbol{g}(\boldsymbol{u})$	$\dfrac{\partial \boldsymbol{g}(\boldsymbol{u})}{\partial \boldsymbol{u}}\dfrac{\partial \boldsymbol{u}}{\partial x}$	$\dfrac{\partial \boldsymbol{g}(\boldsymbol{u})}{\partial \boldsymbol{u}}\dfrac{\partial \boldsymbol{u}}{\partial \boldsymbol{x}}$
$\boldsymbol{f}(\boldsymbol{g}(\boldsymbol{u}))$	$\dfrac{\partial \boldsymbol{f}(\boldsymbol{g})}{\partial \boldsymbol{g}}\dfrac{\partial \boldsymbol{g}(\boldsymbol{u})}{\partial \boldsymbol{u}}\dfrac{\partial \boldsymbol{u}}{\partial x}$	$\dfrac{\partial \boldsymbol{f}(\boldsymbol{g})}{\partial \boldsymbol{g}}\dfrac{\partial \boldsymbol{g}(\boldsymbol{u})}{\partial \boldsymbol{u}}\dfrac{\partial \boldsymbol{u}}{\partial \boldsymbol{x}}$

Transformaciones lineales: $\boldsymbol{x} \in \mathbb{R}^n,\ \boldsymbol{y} = \mathbf{W}\boldsymbol{x},\ \mathbf{W} \in \mathbb{R}^{m\times n},\ \boldsymbol{u} \in \mathbb{R}^m$ constante

$$\frac{\partial y_i}{\partial x_j} = \frac{\partial}{\partial x_j}\sum_k W_{ik}x_k = \sum_k W_{ik}\frac{\partial x_k}{\partial x_j} = W_{ij}$$

$$\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial \boldsymbol{x}} \\ \vdots \\ \frac{\partial y_m}{\partial \boldsymbol{x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix} = \mathbf{W}$$

$$\frac{\partial y_k}{\partial W_{ij}} = \frac{\partial}{\partial W_{ij}}\sum_l W_{kl}x_l = \sum_l x_l\frac{\partial W_{kl}}{\partial W_{ij}} = x_j\mathbb{I}(k=i)$$

$$\frac{\partial y_k}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial y_k}{\partial W_{11}} & \frac{\partial y_k}{\partial W_{21}} & \cdots & \frac{\partial y_k}{\partial W_{m1}} \\ \frac{\partial y_k}{\partial W_{12}} & \frac{\partial y_k}{\partial W_{22}} & \cdots & \frac{\partial y_k}{\partial W_{m2}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_k}{\partial W_{1n}} & \frac{\partial y_k}{\partial W_{2n}} & \cdots & \frac{\partial y_k}{\partial W_{mn}} \end{bmatrix} = \begin{bmatrix} x_1\mathbb{I}(k{=}1) & x_1\mathbb{I}(k{=}2) & \cdots & x_1\mathbb{I}(k{=}m) \\ x_2\mathbb{I}(k{=}1) & x_2\mathbb{I}(k{=}2) & \cdots & x_2\mathbb{I}(k{=}m) \\ \vdots & \vdots & \ddots & \vdots \\ x_n\mathbb{I}(k{=}1) & x_n\mathbb{I}(k{=}2) & \cdots & x_n\mathbb{I}(k{=}m) \end{bmatrix} = \boldsymbol{x}\,\mathrm{onehot}(k)^t$$

$$\boldsymbol{u}^t\frac{\partial \boldsymbol{y}}{\partial \mathbf{W}} = \sum_{k=1}^m u_k\frac{\partial y_k}{\partial \mathbf{W}} = \sum_{k=1}^m u_k\boldsymbol{x}\,\mathrm{onehot}(k)^t = \boldsymbol{x}\sum_{k=1}^m u_k\,\mathrm{onehot}(k)^t = \boldsymbol{x}\boldsymbol{u}^t$$

Transformaciones elementales: $\boldsymbol{x} \in \mathbb{R}^n, \quad \boldsymbol{y} = \boldsymbol{\varphi}(\boldsymbol{x}) \in \mathbb{R}^n, \quad y_i = \varphi(x_i) \quad (i = 1, \ldots, n)$

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} \varphi(x_1) \\ \varphi(x_2) \\ \vdots \\ \varphi(x_n) \end{bmatrix}$$

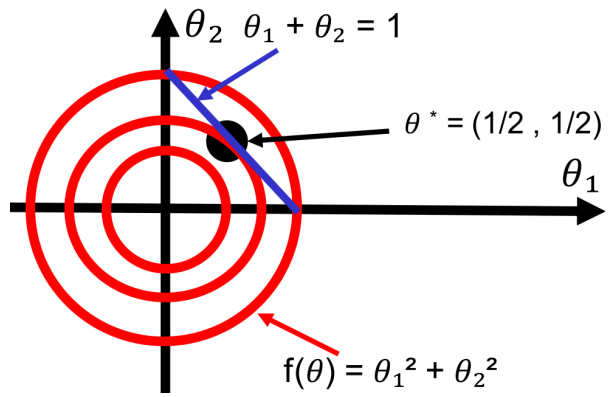
$$\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \begin{bmatrix} \dfrac{\partial y_1}{\partial \boldsymbol{x}} \\ \vdots \\ \dfrac{\partial y_n}{\partial \boldsymbol{x}} \end{bmatrix} = \begin{bmatrix} \dfrac{\partial y_1}{\partial x_1} & \cdots & \dfrac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial y_n}{\partial x_1} & \cdots & \dfrac{\partial y_n}{\partial x_n} \end{bmatrix} = \operatorname{diag}(\varphi'(x_1), \ldots, \varphi'(x_n))$$

7 Optimización con restricciones

Optimización con restricciones: el conjunto de soluciones posibles se restringe con igualdades y desigualdades

$$\boldsymbol{\theta}^* \in \operatorname*{argmin}_{\boldsymbol{\theta}\in\mathcal{C}} \ \mathcal{L}(\boldsymbol{\theta}) \quad \text{con} \quad \mathcal{C} = \{\boldsymbol{\theta}: \ g_j(\boldsymbol{\theta}) \leq 0 : j \in \mathcal{I}, \ h_k(\boldsymbol{\theta}) = 0 : k \in \mathcal{E}\} \in \mathbb{R}^D$$

Ejemplo: Minimizar $\mathcal{L}(\boldsymbol{\theta}) = \theta_1^2 + \theta_2^2$ sujeto a $h(\boldsymbol{\theta}) = 1 - \theta_1 - \theta_2 = 0 \ \rightarrow \ \boldsymbol{\theta}^* = (0.5, 0.5)$



Restricciones de igualdad: se "suben" al objetivo junto con **multiplicadores de Lagrange** y se optimiza sin más

$$L(\boldsymbol{\theta}, \boldsymbol{\lambda}) = \mathcal{L}(\boldsymbol{\theta}) + \sum\nolimits_k \lambda_k h_k(\boldsymbol{\theta})$$

Ejemplo (cont.): $L(\theta_1, \theta_2, \lambda) = \theta_1^2 + \theta_2^2 + \lambda(\theta_1 + \theta_2 - 1)$

$$\frac{\partial L}{\partial(\theta_1, \theta_2, \lambda)} = \begin{pmatrix} 2\theta_1 + \lambda \\ 2\theta_2 + \lambda \\ \theta_1 + \theta_2 - 1 \end{pmatrix}\Bigg|_{(\theta_1^*, \theta_2^*, \lambda^*)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \rightarrow \quad \theta_1^* = \theta_2^* = 0.5$$

Restricciones de igualdad y desigualdad: ignoramos las de desigualdad y se comprueban en la solución